XCS224R Assignment 3

Due Sunday, March 22 at 11:59pm PT.

Guidelines

1. If you have a question about this homework, we encourage you to post your question on our Slack channel, at http://xcs231n-scpd.slack.com/
2. Familiarize yourself with the collaboration and honor code policy before starting work.
3. For the environment setup for the coding problems, please refer to the `README.md` file in the assignment directory.
4. For the coding problems, you must use the packages specified in the provided environment description. Since the autograder uses this environment, we will not be able to grade any submissions which import unexpected libraries.

Submission Instructions

Written Submission: Some questions in this assignment require a written response. For these questions, you should submit a PDF with your solutions online in the online student portal. As long as the PDF is legible and organized, the course staff has no preference between a handwritten and a typeset LaTeX submission. If you wish to typeset your submission and are new to LaTeX, you can get started with the following:

- Type responses only in `submission.tex`.
- Submit the compiled PDF, **not** `submission.tex`.
- Use the commented instructions within the `Makefile` and `README.md` to get started.

Honor code

We strongly encourage students to form study groups. Students may discuss and work on homework problems in groups. However, each student must write down the solutions independently, and without referring to written notes from the joint session. In other words, each student must understand the solution well enough in order to reconstruct it by him/herself. In addition, each student should write on the problem set the set of people with whom s/he collaborated. Further, because we occasionally reuse problem set questions from previous years, we expect students not to copy, refer to, or look at the solutions in preparing their answers. It is an honor code violation to intentionally refer to a previous year's solutions. For SCPD classes, it is also important that students avoid opening pull requests containing their solution code on the shared assignment repositories. More information regarding the Stanford honor code can be found at https://communitystandards.stanford.edu/policies-and-guidance/honor-code.

Writing Code and Running the Autograder

All your code should be entered into the `src/submission/` directory. When editing files in `src/submission/`, please only make changes between the lines containing `### START CODE HERE ###` and `### END CODE HERE ###`. Do not make changes to files outside the `src/submission/` directory.

The unit tests in `src/grader.py` (the autograder) will be used to verify a correct submission. Run the autograder locally using the following terminal command within the `src/` subdirectory:

```
$ python grader.py
```

There are two types of unit tests used by the autograder:

- `basic`: These tests are provided to make sure that your inputs and outputs are on the right track, and that the hidden evaluation tests will be able to execute.

- `hidden`: These unit tests are NOT visible locally. These hidden tests will be run when you submit your code to the Gradescope autograder via the online student portal, and will provide feedback on how many points you have earned. These tests will evaluate elements of the assignment, and run your code with more complex inputs and corner cases. Just because your code passed the basic local tests does not necessarily mean that they will pass all of the hidden tests.

For debugging purposes, you can run a single unit test locally. For example, you can run the test case `3a-0-basic` using the following terminal command within the `src/` subdirectory:

```
$ python grader.py 3a-0-basic
```

Before beginning this course, please walk through the Anaconda Setup for XCS Courses to familiarize yourself with the coding environment. Use the env defined in `src/environment.yml` to run your code. This is the same environment used by the online autograder.

Test Cases

The autograder is a thin wrapper over the python `unittest` framework. It can be run either locally (on your computer) or remotely (on SCPD servers). The following description demonstrates what test results will look like for both local and remote execution. For the sake of example, we will consider two generic tests: `1a-0-basic` and `1a-1-hidden`.

Local Execution - Basic Tests

When a basic test like `1a-0-basic` passes locally, the autograder will indicate success:

```
----- START 1a-0-basic:  Basic test case.
----- END 1a-0-basic [took 0:00:00.000062 (max allowed 1 seconds), 2/2 points]
```

When a basic test like `1a-0-basic` fails locally, the error is printed to the terminal, along with a stack trace indicating where the error occurred:

```
----- START 1a-0-basic:  Basic test case.
<class 'AssertionError'>
{'a': 2, 'b': 1} != None
  File "/Users/grinch/Local_Documents/Software/anaconda3/envs/XCS221/lib/python3.6/unittest/case.py", line 59, in testPartExecutor
    yield
  File "/Users/grinch/Local_Documents/Software/anaconda3/envs/XCS221/lib/python3.6/unittest/case.py", line 605, in run
    testMethod()
  File "/Users/grinch/Local_Documents/SCPD/XCS221/A1/src/graderUtil.py", line 54, in wrapper
    result = func(*args, **kwargs)
  File "/Users/grinch/Local_Documents/SCPD/XCS221/A1/src/graderUtil.py", line 83, in wrapper
    result = func(*args, **kwargs)
  File "/Users/grinch/Local_Documents/SCPD/XCS221/A1/src/grader.py", line 23, in test_0
    submission.extractWordFeatures("a b a"))
  File "/Users/grinch/Local_Documents/Software/anaconda3/envs/XCS221/lib/python3.6/unittest/case.py", line 829, in assertEqual
    assertion_func(first, second, msg=msg)
  File "/Users/grinch/Local_Documents/Software/anaconda3/envs/XCS221/lib/python3.6/unittest/case.py", line 822, in _baseAssertEqual
    raise self.failureException(msg)
----- END 1a-0-basic [took 0:00:00.003809 (max allowed 1 seconds), 0/2 points]
```

This error caused the test to fail.

In this case, start your debugging in line 23 of grader.py.

Remote Execution

Basic and hidden tests are treated the same by the remote autograder, however the output of hidden tests will only appear once you upload your code to GradeScope. Here are screenshots of failed basic and hidden tests. Notice that the same information (error and stack trace) is provided as the in local autograder, now for both basic and hidden tests.

1a-0-basic) Basic test case. (0.0/2.0)

```
<class 'AssertionError'>:  {'a': 2, 'b': 1} != None
  File "/autograder/source/miniconda/envs/XCS221/lib/python3.6/unittest/case.py", line 59, in testPartExecutor
    yield
  File "/autograder/source/miniconda/envs/XCS221/lib/python3.6/unittest/case.py", line 605, in run
    testMethod()
  File "/autograder/source/graderUtil.py", line 54, in wrapper
    result = func(*args, **kwargs)
  File "/autograder/source/graderUtil.py", line 83, in wrapper
    result = func(*args, **kwargs)
  File "/autograder/source/grader.py", line 23, in test_0
    submission.extractWordFeatures("a b a"))
  File "/autograder/source/miniconda/envs/XCS221/lib/python3.6/unittest/case.py", line 829, in assertEqual
    assertion_func(first, second, msg=msg)
  File "/autograder/source/miniconda/envs/XCS221/lib/python3.6/unittest/case.py", line 822, in _baseAssertEqual
    raise self.failureException(msg)
```

Just like in the local autograder, this error caused the test to fail.

Just like in the local autograder, start your debugging in line 23 of grader.py.

1a-1-hidden) Test multiple instances of the same word in a sentence. (0.0/3.0)

```
<class 'AssertionError'>:  {'a': 23, 'ab': 22, 'aa': 24, 'c': 16, 'b': 15} != None
  File "/autograder/source/miniconda/envs/XCS221/lib/python3.6/unittest/case.py", line 59, in testPartExecutor
    yield
  File "/autograder/source/miniconda/envs/XCS221/lib/python3.6/unittest/case.py", line 605, in run
    testMethod()
  File "/autograder/source/graderUtil.py", line 54, in wrapper
    result = func(*args, **kwargs)
  File "/autograder/source/graderUtil.py", line 83, in wrapper
    result = func(*args, **kwargs)
  File "/autograder/source/grader.py", line 31, in test_1
    self.compare_with_solution_or_wait(submission, 'extractWordFeatures', lambda f: f(sentence))
  File "/autograder/source/graderUtil.py", line 183, in compare_with_solution_or_wait
    self.assertEqual(ans1, ans2)
  File "/autograder/source/miniconda/envs/XCS221/lib/python3.6/unittest/case.py", line 829, in assertEqual
    assertion_func(first, second, msg=msg)
  File "/autograder/source/miniconda/envs/XCS221/lib/python3.6/unittest/case.py", line 822, in _baseAssertEqual
    raise self.failureException(msg)
```

This error caused the test to fail.

Start your debugging in line 31 of grader.py.

Finally, here is what it looks like when basic and hidden tests pass in the remote autograder.

1a-0-basic) Basic test case. (2.0/2.0)

1a-1-hidden) Test multiple instances of the same word in a sentence. (3.0/3.0)

0 Overview

Goal

In this assignment you will implement two offline reinforcement learning algorithms: Implicit Q-Learning and Conservative Q-Learning. You'll be experimenting with different hyperparameters and offline datasets that have been provided to you. Data collection is one of the most practical problems with applying Deep RL, and it's a common practice to try different exploration strategies for a given problem. In this assignment, we provide with you Random $\mathcal{E}$-Greedy and Random Network Distillation based exploration strategies for you to compare the quality of the collected data.

You will then have the opportunity to train and tune these agents in a variety of environments in the Simple PointMass physics simulator. We provide the code for generating the offline datasets as part of the assignment. Your objectives are as follows:

1. Implement and train the Implicit Q-Learning and the Conservative Q-Learning algorithms for Offline Reinforcement Learning.
2. Experiment with the key hyperparameters for each algorithm and explore how they affect performance of your RL agents.
3. Analyze differences between the quality of different offline data samples provided to you.

Notice that implementing algorithms may take longer time to finish than analyzing the results.

Codebase

We provide you with offline transitions dataset $\mathcal{D}$. Assuming your replay buffer has been populated with the offline dataset trajectories, your goal is to implement the corresponding reinforcement learning algorithms.

For each problem, we provide you with the files that you need to complete. Sections that need to be filled have been marked with `TODO` tags.

Use of GPT/Codex/Copilot/Any AI code assistant/agent: For the sake of deeper understanding on implementing imitation learning methods, assistance from generative models to write code for this homework is prohibited.

You will submit the entire `submission` directory. To do so run the command below from the `src` directory

```
bash collect_submission.sh
```

or you can choose to zip the folder up manually.

Preliminaries

Environments: In this assignment, we'll be working with the Pointmass environment.

The goal for the Pointmass environment is to navigate a gridworld of varying difficulties: `easy`, `medium` and `hard` to reach the "goal" location. Sample environments for each difficulty have been visualized in Figure 1

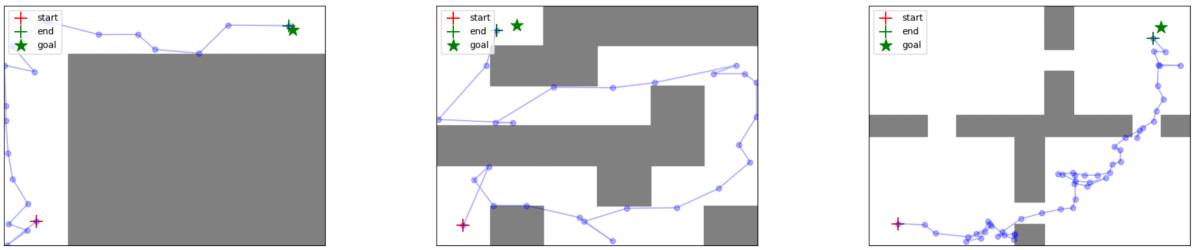

Figure 1: Visualization of the Point mass environment with varying difficulty levels for navigation and reaching the goal.

Offline datasets: For offline RL, we have provided you with a set of exploration strategies which are used to populate the replay buffer. These trajectories then serve as the training dataset for your offline RL algorithms. In this assignment, you'll experiment with two exploration strategies (i) a Random $\mathcal{E}$-Greedy strategy and (ii) Random Network Distillation (RND) algorithm.

The RND algorithm, aims at encouraging exploration by asking the exploration policy to more frequently undertake transitions where the prediction error of a random neural network function is high. Formally, let $f_\theta^*(s')$ be a randomly chosen vector-valued function represented by a neural network. RND trains another neural network, $\hat{f}_\phi(s')$ to match the predictions of $f_\theta^*(s')$ under the distribution of datapoints in the buffer, as shown below:

$$\phi^* = \arg\min_\phi \mathbb{E}_{s,a,s'\sim\mathcal{D}}[\underbrace{\left\|\hat{f}_\phi(s') - f_\theta^*(s')\right\|}_{\mathcal{E}_\phi(s')}] \tag{1}$$

If a transition (s, a, s') is in the distribution of the data buffer, the prediction error $\mathcal{E}_\phi(s')$ is expected to be small. n the other hand, for all unseen state-action tuples it is expected to be large. To utilize this prediction error as a reward bonus for exploration, RND trains two critics – an exploitation critic, $Q_R(s, a)$, and an exploration critic, $Q_\mathcal{E}(s, a)$, where the exploitation critic estimates the return of the policy under the actual reward function and the exploration critic estimates the return of the policy under the reward bonus. In practice, we normalize error before passing it into the exploration critic, as this value can vary widely in magnitude across states leading to poor optimization dynamics.

1 Implicit Q-Learning

In this problem, you'll implement the Implicit Q-Learning (IQL) method for offline RL. The actor update for IQL incorporates the advantage function similar to the actor update in the Advantage-Weighted Actor Critic (AWAC) algorithm. This an advantage-weighted negative log likelihood loss function:

$$L_\pi(\psi) = -\mathbb{E}_{s,a\sim\mathcal{B}}\left[\log \pi_\psi(a \mid s) \exp\left(\frac{1}{\lambda}\mathcal{A}^{\pi_k}(s,a)\right)\right] \tag{2}$$

where $\mathcal{B}$ represents samples sampled from the dataset (behavior policy that was used to collect the data).

The actor update in AWAC corresponds to weighted maximum likelihood, where the targets are updated by reweighting the state-action pairs observed in the current dataset by the predicted advantages from the learned critic. The Q function is learnt with a Temporal Difference (TD) loss. The objective function is given below:

$$\mathbb{E}_D\left[\left(Q(s,a) - (r(s,a) + \gamma\mathbb{E}_{s',a'}\left[Q_{\phi_{k-1}}(s',a')\right])\right)^2\right] \tag{3}$$

IQL modifies the actor critic update to use expectile regression. The expectile ζ of a random variable X is defined as:

$$\arg\min_{m_\zeta} \mathbb{E}_{x\sim X}\left[L_2^\zeta(x - m_\zeta)\right] \tag{4}$$

$$L_2^\zeta(\mu) = |\zeta - \mathbb{1}\{\mu \le 0\}|\mu^2 \tag{5}$$

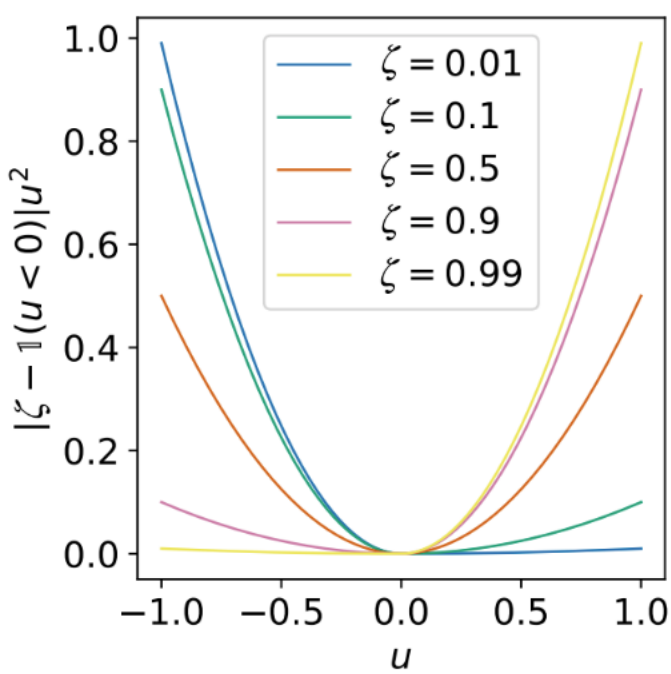

Figure 2: Expectile loss function visualized for different values of ζ.

That is for $\zeta > 0.5$, this asymmetric loss function downweights the contributions of x values smaller than m_ζ, while giving more weights to large values as visualized in Figure 2.

The offline dataset might contain different outcomes from similar states; in standard Q learning, we would want the critic to learn the expected value for any particular state-action pair. IQL, instead of learning the expected value,

learns a given percentile. Our goal is to predict an upper expectile of the TD targets that approximate high values of $r(s,a) + \gamma * Q_\theta(s',a')$ which are present in the support of the offline dataset.

To perform this expectile regression, we need a separate parametric value function V_ϕ. This allows the optimization process to be differentiable, since optimizing with a single parametric q-function implies incorporating the transition dynamics as $s' \sim p(\cdot|s,a)$. (Note the expectation over (s',a') in Equation 3). Finally, the critic is **only** updated with actions that were seen in the offline dataset, and not on any out of distribution (unseen) sampled actions. This leads to the following loss functions:

$$L_V(\phi) = \mathbb{E}_{(s,a)\sim D}\left[L_2^\zeta\left(Q_\theta(s,a) - V_\phi(s)\right)\right] \tag{6}$$

$$L_Q(\theta) = \mathbb{E}_{(s,a,s')\sim D}\left[\left(r(s,a) + \gamma V_\phi\left(s'\right) - Q_\theta(s,a)\right)^2\right] \tag{7}$$

(a) **[20 points (Coding)] Implicit Q-Learning Coding**

Implement the following functions within the below-mentioned files to finish the implementation:

Fill in the `TODO`s in:

- `xcs224r/critics/iql_critic.py`
- `xcs224r/agents/iql_agent.py`

(b) **[8 points (Written)] Experiment and report**

Experiment with $\zeta = 0.2, 0.9$ on the `PointmassEasy-v0` and report the eval average return and standard deviation for both cases. You can look at the final eval trajectories under the saved logs to qualitatively observe your agent's performance. The code would run for 50,000 iterations and would roughly take at most an hour to finish. Attach the `eval_last_traj.png` image denoting your agent's last trajectory for each run. Which ζ value performs better and why?

```
python run_iql.py --env_name PointmassEasy-v0 \
--exp_name iql_zeta_{enter_your_zeta}_rnd --use_rnd \
--num_exploration_steps=20000 \
--unsupervised_exploration \
--awac_lambda=1 \
--iql_expectile={enter_your_zeta}
```

(c) **[8 points (Written)] Compare the learnt policies**

Compare the learnt policies for the RND and Random exploration on the `PointmassMedium-v0` environment using the best ζ value from the previous runs. Report the eval average return and standard deviation. What does the performance gap between the two exploration strategies indicate about the collected data in case of offline RL?

Remove the `--use_rnd` flag to run with random exploration:

```
python run_iql.py --env_name PointmassMedium-v0 \
--exp_name iql_zeta_{enter_your_zeta}_random \
--num_exploration_steps=20000 \
--unsupervised_exploration \
--awac_lambda=1 \
--iql_expectile={enter_your_zeta}
```

Attach the `eval_last_traj.png` image denoting your agent's last trajectory for each run.

2 Conservative Q-Learning

In this problem, you'll be implementing the Conservative Q-Learning (CQL) algorithm. The goal of CQL is to prevent overestimation of the policy value. The overall CQL objective is given by the standard TD error objective augmented with the CQL regularizer weighted by α : $\alpha \left[\frac{1}{N}\sum_{i=1}^{N}\left(\log\left(\sum_a \exp\left(Q\left(s_i, a\right)\right)\right) - Q\left(s_i, a_i\right)\right)\right]$.

(a) **[10 points (Coding)] Conservative Q-Learning Coding**

Fill in the `TODO`s in the following files:

- `xcs224r/critics/cql_critic.py`

(b) **[8 points (Written)] Experiment and report**

Once you've filled in all of the `TODO` commands, you should be able to run CQL with the following command -

```
python run_cql.py --env_name PointmassHard-v0 \
    --exp_name cql_alpha_{enter_your_alpha}_rnd \
    --use_rnd --unsupervised_exploration \
    --offline_exploitation --cql_alpha={enter_your_alpha}
```

You can look at the final eval trajectories under the saved logs to qualitatively observe your agent's performance. The code would run for 50,000 iterations and would roughly take about an hour to finish.

Attach the `eval_last_traj.png` image denoting your agent's last trajectory for each run.

Experiment with $\alpha = 0, 0.1$ on the `PointmassHard-v0` and report the Eval average return and standard deviation for both cases on the PointmassHard-v0 environment.

What does $\alpha = 0.0$ signify in terms of the algorithm?

Explain the difference in obtained performance gap based on α (Performance may vary by different random seeds, please run the experiment 3 times and report all runs).

(c) **[6 points (Written)] Compare the learnt policies**

Compare the learnt policies for the RND and Random exploration on the `PointmassHard-v0` environment with $\alpha = 0.1$ by reporting the eval average return and standard deviation. Remove the `--use_rnd` flag to run with random exploration:

```
python run_cql.py --env_name PointmassHard-v0 \
    --exp_name cql_alpha_0.1_random \
    --unsupervised_exploration \
    --offline_exploitation --cql_alpha=0.1
```

Attach the `eval_last_traj.png` image denoting your agent's last trajectory for each run (Performance may vary by different random seeds, please run the experiment 3 times and report all runs).